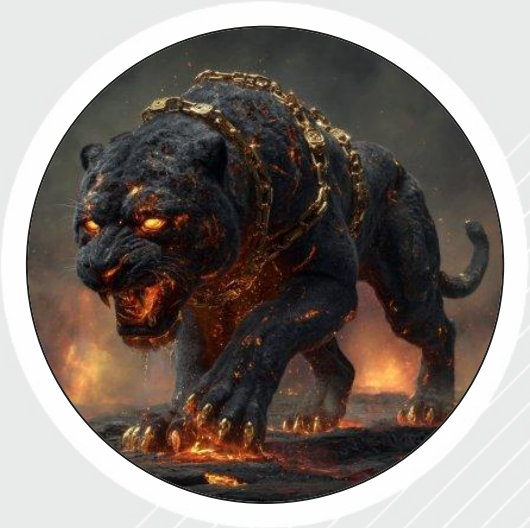
SPYWOLF

Security Audit Report



Audit prepared for

HellBurn

Completed on

March 12, 2026

SPYWOLF.CO
@SPYWOLFNETWORK
@SPYWOLFNETWORK

TABLE OF CONTENTS

Project Information 01

Scope of Audit 02

Audit Methodology 03

Findings and Vulnerability Analysis 04

Conclusion 05

About SPYWOLF 06

Disclaimer 07

PROJECT INFORMATION

Project Name: HellBurn Protocol (HBURN)
Website: https://hellburn.win
Blockchain: Ethereum Mainnet
Solidity Version: ^0.8.24
Framework: OpenZeppelin Contracts (ERC20, ERC20Burnable, ReentrancyGuard, Pausable, SafeERC20)
External Dependencies: Uniswap V3 (SwapRouter, NonfungiblePositionManager, WETH9)
Token Standard: ERC-20
Token Name / Symbol: HellBurn / HBURN
Maximum Supply: 1,000,000,000,000 HBURN (1 Trillion, hard-capped)
Minting Mechanism: Genesis-only. Minting is permanently disabled after the 28-day genesis phase concludes. No admin, owner, or governance function can re-enable minting.

Protocol Overview

HellBurn is a deflationary DeFi protocol built within the TitanX and DragonX ecosystem on Ethereum. Participants permanently destroy TitanX and DragonX tokens in exchange for HBURN tokens and ETH yield. The protocol is designed with zero admin keys, no owner privileges, and no upgrade paths, making all core mechanics fully immutable and trustless once deployed.

The Genesis Phase is a 28-day fair-launch window during which users burn TitanX to mint HBURN at declining bonus rates across four weekly tiers, ranging from a 1:1 ratio with 15% bonus in week one down to a 0.85:1 ratio with no bonus in week four. Of incoming TitanX, 35% is permanently burned, 35% goes to the DragonX Vault, 22% to the treasury for long-duration staking, and 8% is retained as a Genesis Fund for liquidity creation. Of the HBURN minted, 3% is reserved for the liquidity pool while 97% goes to the user, split between 25% immediate release and 75% subject to 28-day linear vesting with independent per-tranche accounting.

When genesis concludes, anyone may trigger the fair-launch sequence. The Genesis Fund TitanX is swapped to WETH via Uniswap V3, paired with the HBURN LP reserve to create a full-range liquidity position, and the resulting LP-NFT is permanently locked in the contract with no withdrawal function.

The Staking System allows HBURN holders to lock tokens for 28 to 3,500 days, earning ETH yield proportional to share weight. Shares are determined by a time bonus scaling from 1.0x to 3.5x, a fuel bonus from burning TitanX or DragonX into the stake up to 1.5x, and a loyalty bonus of 1.1x for consecutive re-stakes. Three consecutive re-stakes grant permanent Phoenix Status with an additional 5% compounding bonus. Early unstaking after 50% maturity incurs a linearly decreasing penalty, half of which is burned and half retained by the contract. Stakers are classified into five tiers from Bronze at 28 days to Diamond at 3,500 days.

The Burn Epochs System runs continuous 8-day competitive cycles where participants burn TitanX at 1x weight and DragonX at 2x weight. A streak multiplier increments by 0.2x per consecutive epoch from a 1.2x base up to 3.0x maximum, resetting on any missed epoch. Upon finalization, 80% of deposited ETH is distributed proportionally to participants and 20% is forwarded to Buy-and-Burn.

The Buy-and-Burn contract accumulates ETH from Burn Epochs and LP trading fees, purchases HBURN on Uniswap V3, and permanently removes it from circulation. It is callable by anyone with no access restrictions, operating as a public good that continuously applies deflationary pressure.

SCOPE OF AUDIT

Audit Overview

Audit Type: Manual Smart Contract Security Review
Audit Date: March 2026
Audited By: Independent Security Auditor
Review Method: Line-by-line manual source code analysis with scenario-based exploit simulation
Language: Solidity ^0.8.24
Total Lines of Code: Approximately 1,537
Test Suite Provided: HellBurn_test.js (1,093 lines, Hardhat framework with mock Uniswap V3 infrastructure)

Contracts in Scope

Contract	File	Lines	Description
HellBurnToken	HellBurnToken.sol	54	ERC-20 token with genesis-only minting and permanent mint-disable mechanism
GenesisBurn	GenesisBurn.sol	515	28-day fair launch with TitanX burn-to-mint, per-tranche vesting, automated LP creation, and fee collection
HellBurnStaking	HellBurnStaking.sol	372	Time-locked HBURN staking with ETH yield distribution, fuel mechanic, loyalty system, and Phoenix bonuses
BurnEpochs	BurnEpochs.sol	379	Competitive 8-day burn cycles with streak multipliers, per-epoch ETH tracking, and carry-over mechanics
BuyAndBurn	BuyAndBurn.sol	106	ETH accumulator that purchases HBURN via Uniswap V3 and permanently removes it from circulation

Interfaces in Scope

Interface	File	Lines	Description
ITitanX	ITitanX.sol	25	Minimal interface for TitanX token and staking interactions
ISwapRouter	ISwapRouter.sol	31	Uniswap V3 SwapRouter interface with WETH wrapper interface
INonfungiblePosition Manager	INonfungiblePosition Manager.sol	55	Uniswap V3 NonfungiblePositionManager interface for LP position minting and fee collection

AUDIT METHODOLOGY

The review was conducted through manual line-by-line analysis of all Solidity source code, simulating the methodology of an expert smart contract security auditor. No automated scanning tools were used as the primary analysis method. The review process followed four stages:

Stage 1: Architecture Review. Mapped all contracts, their inheritance chains, cross-contract dependencies, and external protocol integrations. Identified all entry points, privileged roles, and trust boundaries.

Stage 2: Function-Level Analysis. Examined every public and external function for correctness of access control, state mutation ordering, arithmetic precision, reentrancy exposure, and adherence to the checks-effects-interactions pattern.

Stage 3: Scenario Simulation. Constructed multi-step attack scenarios across the following categories: economic exploits through share or reward manipulation, front-running and MEV extraction, denial-of-service through gas griefing or state bloat, cross-contract state inconsistencies, and token accounting discrepancies between minted, burned, locked, and circulating supply.

Stage 4: Integration Review. Evaluated the correctness of interactions with Uniswap V3 contracts including swap execution, pool initialization, LP position minting, and fee collection. Assessed whether mock-based test coverage adequately represents mainnet behavior.

Methodology for Severity Rating

Each issue was assigned a severity level based on two dimensions:

Severity	Impact	Likelihood	Description
Critical	Direct loss of funds or total system compromise	High	Must be fixed before deployment
High	Potential loss of funds or systemic malfunction	Medium–High	Priority for immediate remediation
Medium	Logic or validation flaws that can cause local issues	Medium	Should be fixed before mainnet
Low	Minor logic errors, inefficiencies, or incomplete patterns	Low	Recommended improvement

Severity was determined by **impact × exploitability**, emphasizing how realistic an attacker scenario is under the given system assumptions.

FINDINGS AND VULNERABILITY ANALYSIS

Summary of Findings

ID	Title	Severity	Status
C-01	addFuel Does Not Reset ETH Debt; Retroactive Yield Theft	**Critical**	Open
C-02	collectLPFees Accepts Arbitrary Recipient; LP Fee Theft	**Critical**	Open
H-01	ETH Permanently Lost When Received With Zero Active Stakers	**High**	Open
H-02	Precision Loss in _calculateSqrtPriceX96 Mis-prices Initial LP	**High**	Open
H-03	Unbounded Vesting Tranche Array; Gas Griefing on claimVested()	**High**	Open
M-01	No Enforcement of Reasonable Slippage on endGenesis Swap	**Medium**	Open
M-02	block.timestamp Deadlines Provide No Transaction Expiry Protection	**Medium**	Open
M-03	Penalty-Retained HBURN Accumulates With No Distribution Path	**Medium**	Open
M-04	finalizeUpTo Reverts Entirely if Single ETH Transfer Fails	**Medium**	Open
M-05	HBURN Burned to Dead Address; totalSupply() Never Decreases	**Medium**	Open
L-01	ETH Deposit Allocated to Wrong Epoch at Boundary Timestamp	**Low**	Open
L-02	batchClaimRewards Silently Skips Invalid Epochs With No Feedback	**Low**	Open
L-03	Dust-Level Fuel Burns Still Increment Shares Due to Floor of 1	**Low**	Open
L-04	No Validation That firstEpochStart Is in the Future	**Low**	Open
L-05	Immutable Addresses Prevent Recovery if Redeployment Needed	**Low**	Open
I-01	Genesis Fund Computed as Remainder Rather Than Direct Percentage	**Info**	Open
I-02	Test Suite Uses Mocks That Don't Replicate Real Uniswap Behavior	**Info**	Open
I-03	completedStakes Increments on Early Exits, Not Just Mature Stakes	**Info**	Open
I-04	onERC721Received Accepts Any NFT, Not Just the LP Position	**Info**	Open

FINDINGS

Critical Risk

C-01: addFuel Does Not Reset ETH Debt — Retroactive Yield Theft From Other Stakers (1)

Component: HellBurnStaking / _addFuel, _pendingETH

Description: When a user adds fuel to an active stake, the _addFuel function removes the old shares from totalShares, recalculates a higher share count based on the new fuel bonus, and adds the new shares back. However, the function never updates stakeETHDebt[stakeId]. The pending ETH reward formula in _pendingETH computes (newShares * (ethPerShare - originalDebt)) / 1e18, which means the boosted shares are retroactively applied to all ETH distributions that occurred since the stake was created. This allows the fueling user to claim ETH yield they never earned, directly draining rewards that belong to other stakers. If enough ETH is drained this way, later stakers who attempt to claim will find the contract balance insufficient to cover their legitimate rewards.

```
// HellBurnStaking.sol — _addFuel() lines 299-309
function _addFuel(uint256 stakeId, address token, uint256 amount, uint256 weight) internal {
    // ... burns token, calculates fuelIncrement ...

    // Remove old shares, recalculate with new fuel
    totalShares -= s.shares;
    s.fuelBonus = s.fuelBonus + fuelIncrement;
    if (s.fuelBonus > MAX_FUEL_BONUS) s.fuelBonus = MAX_FUEL_BONUS;

    uint256 numDays = (s.endTime - s.startTime) / 1 days;
    uint256 timeBonus = _timeBonus(numDays);
    s.shares = (s.amount * timeBonus * s.fuelBonus * s.loyaltyBonus) / (BASIS * FUEL_BASIS * BASIS);

    totalShares += s.shares;
    // ❌ stakeETHDebt[stakeId] is NEVER updated — old debt persists with new shares
}

// _pendingETH() — the reward formula that gets exploited
function _pendingETH(uint256 stakeId) internal view returns (uint256) {
    Stake storage s = stakes[stakeId];
    if (!s.active || totalShares == 0) return 0;
    return (s.shares * (ethPerShare - stakeETHDebt[stakeId])) / 1e18;
    //      ^^^^^^^^ new inflated shares applied to full historical ethPerShare gap
}
```

Impact:

- Direct theft of ETH from other stakers' pending rewards
- Contract can become insolvent — total owed rewards exceed actual ETH balance
- Later claimants receive less than they are owed or transactions revert entirely
- Exploitable repeatedly with minimal cost since even dust-level fuel burns trigger the share recalculation

FINDINGS

Critical Risk

C-01: addFuel Does Not Reset ETH Debt — Retroactive Yield Theft From Other Stakers (2)

Likelihood: High. No special privileges or unlikely preconditions are required. Any staker can exploit this by simply calling addFuelTitanX or addFuelDragonX after ETH distributions have occurred.

Simulation Steps:

1. Alice and Bob each start a stake with equal amounts and duration, resulting in 100 shares each. totalShares = 200.
2. 20 ETH is sent to the staking contract. ethPerShare increases such that each user is owed 10 ETH.
3. Alice calls addFuelTitanX with a small amount, boosting her shares from 100 to 150. Her stakeETHDebt remains unchanged.
4. Alice's _pendingETH now returns 150 * (ethPerShare - originalDebt) / 1e18 = 15 ETH instead of the correct 10 ETH.
5. Alice calls endStake and receives 15 ETH. Only 5 ETH remains in the contract.
6. Bob calls endStake and receives only 5 ETH instead of his rightful 10 ETH. 5 ETH has been stolen.

Status: Open; immediate remediation required before deployment.

Category: Reward Accounting / Share Manipulation

Recommendation:

- Before modifying shares in _addFuel, settle the stake's pending ETH by either paying it out to the user or accumulating it in a realizedETH[stakeId] mapping.
- After recalculating shares, reset stakeETHDebt[stakeId] = ethPerShare so the new shares only earn yield from distributions that occur after the fuel was added.
- Update _pendingETH to include any previously realized balance: return realizedETH[stakeId] + (s.shares * (ethPerShare - stakeETHDebt[stakeId])) / 1e18.

FINDINGS

Critical Risk

C-02: collectLPFees Accepts Arbitrary Recipient Address — LP Fee Theft (1)

Component: GenesisBurn / collectLPFees

Description: The collectLPFees function collects accumulated trading fees from the protocol-owned Uniswap V3 LP position. The HBURN portion of the fees is correctly sent to the dead address for permanent removal. However, the WETH portion is unwrapped to ETH and forwarded to whatever address the caller passes as the buyAndBurn parameter. The only validation performed is a zero-address check. Since the function is callable by anyone and there is no verification that the supplied address is the actual BuyAndBurn contract, any attacker can call collectLPFees(attackerAddress) and receive the entire accumulated WETH fee balance as raw ETH. This is a direct and permissionless theft vector that can be executed every time fees accumulate in the LP position.

```
// GenesisBurn.sol — collectLPFees() lines 270-301
function collectLPFees(address buyAndBurn) external nonReentrant {
    if (!lpCreated) revert LPNotCreated();
    require(buyAndBurn != address(0), "zero addr");
    //      ^^^^^^^^^ only check — any non-zero address is accepted

    (uint256 amount0, uint256 amount1) = positionManager.collect(
        INonfungiblePositionManager.CollectParams({
            tokenId: lpTokenId,
            recipient: address(this),
            amount0Max: type(uint128).max,
            amount1Max: type(uint128).max
        })
    );

    bool hburnIsToken0 = address(hburn) < address(weth);
    uint256 hburnFees = hburnIsToken0 ? amount0 : amount1;
    uint256 wethFees = hburnIsToken0 ? amount1 : amount0;

    // HBURN fees — correctly burned
    if (hburnFees > 0) {
        IERC20(address(hburn)).safeTransfer(DEAD_ADDRESS, hburnFees);
    }

    // WETH fees — sent to ATTACKER-CONTROLLED address as raw ETH
    if (wethFees > 0) {
        weth.withdraw(wethFees);
        (bool sent,) = buyAndBurn.call{value: wethFees}("");
        //             ^^^^^^^^^ caller-supplied, unvalidated destination
        if (!sent) revert TransferFailed();
    }

    emit LPFeesCollected(hburnFees, wethFees);
}
```

Impact:

- Complete theft of all accumulated WETH LP trading fees, every time fees are collected
- Continuous drain — the attacker can monitor fee accrual and front-run legitimate callers to extract fees as soon as they become non-trivial
- The HBURN deflationary mechanism through LP fee burning remains intact, but the ETH that should fuel BuyAndBurn is permanently diverted, eliminating a core source of buy-and-burn pressure
- Zero cost to exploit beyond gas fees

FINDINGS

Critical Risk

C-02: collectLPFees Accepts Arbitrary Recipient Address — LP Fee Theft (2)

Likelihood: Critical. No preconditions, no privileges, and no timing constraints. The LP position generates fees on every trade in the HBURN/WETH pool. An attacker simply needs to call the function with their own address. Bots can automate this to extract fees within the same block they accumulate.

Simulation Steps:

1. Genesis completes and LP is created. Trading begins in the HBURN/WETH pool, generating fees.
2. The attacker deploys a minimal contract with a receive() external payable {} fallback, or simply uses their own EOA.
3. The attacker calls genesis.collectLPFees(attackerAddress).
4. The position manager returns accumulated fees to the GenesisBurn contract.
5. HBURN fees are sent to the dead address as expected.
6. WETH fees are unwrapped and the resulting ETH is sent directly to the attacker's address.
7. The attacker repeats this every time meaningful fees accumulate.

Status: Open; immediate remediation required before deployment.

Category: Access Control / Unvalidated Parameter

Recommendation:

- Remove the buyAndBurn parameter entirely. Instead, store the BuyAndBurn contract address as an immutable state variable set in the constructor.
- The contract already has access to hburn.buyAndBurnContract() through the HellBurnToken reference, which stores this address as an immutable.
- Alternatively, accept an immutable buyAndBurn address in the GenesisBurn constructor and validate against it, consistent with how other contracts in the protocol handle trusted addresses.

FINDINGS

High Risk

H-01: ETH Permanently Lost When Received With Zero Active Stakers (1)

Component: HellBurnStaking / _distributeETH, receive

Description: The _distributeETH function is called whenever ETH is sent to the staking contract, either through the receive fallback or the depositETH function. If totalShares is zero at the time of receipt (meaning no stakes are currently active) the function silently returns without recording the deposit in any way. The ETH is accepted by the contract but never tracked, never allocated to any future distribution, and never made recoverable. There is no carry-over mechanism, no unallocated balance variable, and no admin withdrawal function. The ETH becomes permanently locked in the contract with no path to recovery. This can occur during normal protocol operation: after genesis but before the first stake is created, during any window where all existing stakes have been ended, or if an external protocol or automation sends ETH at an unfortunate moment. Unlike BurnEpochs, which explicitly handles this scenario through its carryOverETH mechanism, the staking contract has no equivalent safeguard.

```
// HellBurnStaking.sol — _distributeETH() lines 314-319
function _distributeETH(uint256 amount) internal {
    if (totalShares > 0 && amount > 0) {
        ethPerShare += (amount * 1e18) / totalShares;
        totalETHReceived += amount;
        emit ETHRewardsReceived(amount);
    }
    // ❌ When totalShares == 0: no revert, no tracking, no event
    //    ETH silently accepted and permanently locked
}

// receive() — accepts ETH unconditionally
receive() external payable {
    _distributeETH(msg.value);  // if totalShares == 0, ETH is lost
}

// depositETH() — same path
function depositETH() external payable {
    _distributeETH(msg.value);  // same silent loss
}
```

Impact:

- Permanent loss of ETH sent to the contract when no stakes are active
- The totalETHReceived counter does not increment for lost ETH, making the loss invisible to off-chain monitoring that relies on contract state
- No event is emitted, so the loss is also invisible to event-based tracking systems
- Can affect protocol-level ETH flows if BurnEpochs, LP fee collection, or any other contract is configured to send ETH to the staking contract during a zero-staker window
- The loss is cumulative and irreversible — every ETH sent during zero-share periods is permanently locked

FINDINGS

High Risk

H-01: ETH Permanently Lost When Received With Zero Active Stakers (2)

Likelihood: High. The window between genesis ending and the first stake being created is a natural zero-share period. Additionally, if the protocol experiences a period of low activity where all stakers have exited, any ETH arriving from external sources during that gap is lost. The BurnEpochs contract sends ETH to BuyAndBurn rather than staking directly, but LP fee collection and manual deposits are realistic sources.

Simulation Steps:

1. Genesis ends. No user has staked yet. totalShares = 0.
2. A user or external contract calls depositETH() with 5 ETH, or sends ETH directly to the staking contract address.
3. _distributeETH(5 ETH) is called. The condition totalShares > 0 is false. The function returns without any action.
4. The contract's ETH balance is now 5 ETH, but totalETHReceived is still 0, ethPerShare is still 0, and no event was emitted.
5. Alice starts a stake. stakeETHDebt[0] = ethPerShare = 0.
6. Alice's _pendingETH returns 0 because ethPerShare never reflected the 5 ETH.
7. The 5 ETH sits in the contract permanently with no mechanism to distribute or recover it.

Status: Open; strongly recommended to remediate before deployment.

Category: Fund Loss / Missing Safety Check

Recommendation:

- Introduce an unallocatedETH state variable that captures ETH received when totalShares == 0, mirroring the carryOverETH pattern used in BurnEpochs.
- When a new stake is created in _startStake, check if unallocatedETH > 0 and totalShares > 0 after adding the new shares, then flush the unallocated balance into ethPerShare.
- Alternatively, revert on receive() and depositETH() when totalShares == 0 to prevent ETH from entering the contract when it cannot be distributed. This is simpler but may break composability with contracts that expect sends to succeed.

FINDINGS

High Risk

H-02: Precision Loss in _calculateSqrtPriceX96 Can Mis-price the Initial LP Position (1)

Component: GenesisBurn / _calculateSqrtPriceX96, _sqrt, _createLiquidityPool

Description: When the genesis phase concludes and liquidity is created, the contract must compute the initial sqrtPriceX96 value that Uniswap V3 uses to set the starting price of the HBURN/WETH pool. The mathematically correct formula is sqrt(amount1 / amount0) * 2^96, which is equivalent to sqrt(amount1 * 2^192 / amount0). However, the implementation splits this into two independent integer square roots — sqrt(amount1) and sqrt(amount0); computed separately via the Babylonian method, then combines them as sqrt(amount1) * 2^96 / sqrt(amount0). Integer square root truncates the result, and performing two independent truncations compounds the rounding error. When the two token amounts have vastly different magnitudes; which is the expected case since the LP pairs potentially trillions of HBURN against a small amount of WETH; the relative error becomes significant. An incorrect initial sqrtPriceX96 means the pool launches at a price that doesn't reflect the true ratio of deposited assets. Arbitrageurs monitoring pool creation events will immediately exploit any mispricing by trading against the LP position, extracting value at the expense of the protocol's permanent liquidity.

```
// GenesisBurn.sol — _calculateSqrtPriceX96() lines 431-450
function _calculateSqrtPriceX96(uint256 amount0, uint256 amount1)
    internal pure returns (uint160)
{
    uint256 sqrtAmount1 = _sqrt(amount1);  // ❌ truncation #1
    uint256 sqrtAmount0 = _sqrt(amount0);  // ❌ truncation #2

    require(sqrtAmount0 > 0, "zero amount0");

    // Two truncated values combined — error compounds
    return uint160((sqrtAmount1 * (1 << 96)) / sqrtAmount0);
    //              ^^^^^^^^^^^                ^^^^^^^^^^^
    //              rounded down               rounded down
    //              then multiplied by 2^96 — amplifies error
}

/// @dev Babylonian square root — always rounds DOWN
function _sqrt(uint256 x) internal pure returns (uint256) {
    if (x == 0) return 0;
    uint256 z = (x + 1) / 2;
    uint256 y = x;
    while (z < y) {
        y = z;
        z = (x / z + z) / 2;
    }
    return y;  // floor(sqrt(x))
}
```

Impact:

- The HBURN/WETH pool launches at an incorrect price that deviates from the intended ratio of WETH_received / HBURN_reserved
- Arbitrageurs extract value from the mispricing immediately upon pool creation, reducing the effective liquidity that the protocol permanently locks
- The loss is one-time but permanent…the extracted value can never be recovered since the LP-NFT is locked in the contract
- The error magnitude scales with the disparity between token amounts. With trillions of HBURN (1e30 range) versus fractions of WETH (1e16-1e18 range), the relative error of the split-sqrt approach can reach basis-point or even percentage-level deviations
- amount0Min and amount1Min are both set to 0 in the LP mint call, providing no backstop against depositing at a distorted price

FINDINGS

High Risk

H-02: Precision Loss in _calculateSqrtPriceX96 Can Mis-price the Initial LP Position (2)

Likelihood: Medium-High. The mispricing is guaranteed to some degree due to the mathematical properties of integer truncation. The exploitability depends on the magnitude of the error relative to gas costs. Given the expected token magnitude disparity between HBURN and WETH, the error is likely to be non-trivial and profitable for arbitrage bots that monitor createAndInitializePoolIfNecessary events.

Simulation Steps:

1. Genesis concludes with lpReserveHBURN = 27,600,000 * 1e18 (27.6M HBURN) and genesisFundTitanX swaps to wethReceived = 0.8 * 1e18 (0.8 WETH).
2. Assuming HBURN is token0: amount0 = 27,600,000e18, amount1 = 0.8e18.
3. Correct sqrtPriceX96 = sqrt(0.8e18 / 27,600,000e18) * 2^96 = sqrt(2.8986e-8) * 2^96.
4. The split-sqrt method computes _sqrt(0.8e18) = 894,427,190 and _sqrt(27,600,000e18) = 166,132,477,257, each truncated independently.
5. Result: 894,427,190 * 2^96 / 166,132,477,257. The two independent truncations compound into an output that deviates from the mathematically correct value.
6. Pool is initialized at the incorrect price. An arbitrageur immediately swaps against the position, pocketing the difference and permanently reducing the LP's depth.

Status: Open; strongly recommended to remediate before deployment.

Category: Arithmetic Precision / Integration Risk

Recommendation:

- Replace the split-sqrt approach with a single-sqrt computation that preserves precision by scaling before taking the root
- Use OpenZeppelin's Math.sqrt() or Solady's FullMath.mulDiv and sqrt for overflow-safe, precision-maximizing arithmetic.
- Alternatively, use Uniswap's own TickMath and SqrtPriceMath libraries which are specifically designed for computing sqrtPriceX96 values correctly across all magnitude ranges.
- As an additional safeguard, set non-zero amount0Min and amount1Min in the LP mint call to ensure that if the computed price is significantly off, the transaction reverts rather than depositing at a distorted ratio.

FINDINGS

High Risk

H-03: Unbounded Vesting Tranche Array Growth — Gas Griefing on claimVested() (1)

Component: GenesisBurn / burn, claimVested, claimableAmount

Description: Every call to the burn function during the genesis phase pushes a new VestingTranche struct into the caller's vestingTranches array. There is no upper bound on the number of tranches a single address can accumulate. The claimVested function iterates over the entire array to calculate and settle all pending vesting claims, as does the view function claimableAmount. If a user (intentionally or through automated interaction) executes a large number of small burns, the array grows to a size where iterating it exceeds the block gas limit. At that point, claimVested becomes permanently uncallable for that address, and all vested HBURN held by the contract on behalf of that user is permanently locked. While this is self-inflicted in the case of a normal user making many small burns, there is a more concerning vector: any contract or script that interacts with genesis on behalf of users could inadvertently create this condition. Additionally, since the burn function only requires a TitanX approval and a non-zero amount, the minimum burn can be as small as 1 wei of TitanX, making array inflation extremely cheap.

```
// GenesisBurn.sol — burn() lines 196-201
// Each call pushes a new tranche with no array size limit
vestingTranches[msg.sender].push(VestingTranche({
    amount: vestedAmount,
    vestingStart: block.timestamp,
    claimed: 0
}));
// ❌ No cap on array length — grows by 1 per burn call

// GenesisBurn.sol — claimVested() lines 214-230
function claimVested() external nonReentrant {
    uint256 totalClaimable = 0;
    VestingTranche[] storage tranches = vestingTranches[msg.sender];

    for (uint256 i = 0; i < tranches.length; i++) {  // ❌ unbounded loop
        uint256 trancheClaimable = _trancheClaimable(tranches[i]);
        if (trancheClaimable > 0) {
            tranches[i].claimed += trancheClaimable;
            totalClaimable += trancheClaimable;
        }
    }

    if (totalClaimable == 0) revert NothingToClaim();
    IERC20(address(hburn)).safeTransfer(msg.sender, totalClaimable);
    emit VestingClaimed(msg.sender, totalClaimable);
}

// GenesisBurn.sol — claimableAmount() lines 304-311
function claimableAmount(address user) external view returns (uint256) {
    uint256 total = 0;
    VestingTranche[] storage tranches = vestingTranches[user];
    for (uint256 i = 0; i < tranches.length; i++) {  // ❌ same unbounded loop
        total += _trancheClaimable(tranches[i]);
    }
    return total;
}
```

Impact:

- Permanent locking of vested HBURN for any user whose tranche array exceeds the gas limit threshold for iteration
- The claimableAmount view function also becomes uncallable, preventing the user from even checking their vesting status
- At approximately 50,000 gas per tranche iteration (storage reads plus arithmetic), the block gas limit of 30M gas would be exceeded at roughly 600 tranches — achievable with 600 minimum-amount burns during the 28-day genesis window
- Cost of attack is minimal: 600 burns of 1 wei TitanX each costs virtually nothing in token terms, only gas for the 600 transactions
- Vested tokens locked this way are permanently irrecoverable since the contract has no admin function or alternative claim path

FINDINGS

High Risk

H-03: Unbounded Vesting Tranche Array Growth — Gas Griefing on claimVested() (2)

Likelihood: Medium. Self-griefing requires deliberate or careless behavior from the user. However, aggregator contracts, batch-burning interfaces, or automated DCA strategies that call burn in small increments over the 28-day genesis could realistically create this condition. The low cost of each burn call makes intentional griefing trivially affordable.

Simulation Steps:

1. During the genesis phase, an attacker or careless user approves TitanX to the GenesisBurn contract.
2. The user calls burn(1) in a loop 700 times across multiple transactions. Each call pushes a new VestingTranche to the array. Total TitanX cost: 700 wei (negligible).
3. Each tranche has a vestedAmount near zero, but the array length is now 700.
4. The user calls claimVested(). The function attempts to iterate all 700 tranches. Gas consumption exceeds the block gas limit. Transaction reverts.
5. Any non-trivial burns the user made in the same address are also trapped — the function must iterate the entire array and there is no way to skip or paginate.
6. The vested HBURN remains in the GenesisBurn contract permanently. No alternative claim path exists.

Status: Open; strongly recommended to remediate before deployment.

Category: Denial of Service / Unbounded Iteration

Recommendation:

- Implement paginated claiming that allows the user to specify a range of tranches to process.
- Additionally, enforce a minimum burn amount in the burn function (e.g., require(titanXAmount >= 1e18, "below minimum")) to make array inflation economically impractical.
- Consider consolidating fully-claimed tranches by swapping the last element into the claimed slot and popping, reducing the effective array length over time.

FINDINGS

Medium Risk

M-01: No Enforcement of Reasonable Slippage on endGenesis LP Creation Swap (1)

Component: GenesisBurn / endGenesis, _createLiquidityPool

Description: When the genesis phase concludes, anyone can call endGenesis(uint256 minWETHOut) to trigger the TitanX-to-WETH swap and subsequent LP creation. The minWETHOut parameter is passed directly to the Uniswap V3 router's amountOutMinimum field with no on-chain floor or sanity check. This means the very first caller can pass minWETHOut = 0, executing the entire Genesis Fund swap with zero slippage protection. Since the function is intentionally designed to be callable by anyone (a "public good" pattern), there is no guarantee that the first caller will use a reasonable value. In practice, MEV bots actively monitor for large unprotected swaps and will sandwich the transaction; front-running with a buy to inflate the price, letting the genesis swap execute at a worse rate, then back-running with a sell to capture the difference. The Genesis Fund represents 8% of all TitanX contributed during the entire 28-day genesis, so the amount at risk could be substantial. It's worth noting that the BuyAndBurn contract correctly enforces minHBURNOut != 0 (the C-02 audit fix), but the same discipline was not applied here.

```
// GenesisBurn.sol; endGenesis() lines 246-259
function endGenesis(uint256 minWETHOut) external nonReentrant {
    if (block.timestamp <= genesisEnd) revert GenesisNotYetEnded();
    if (genesisEnded) revert GenesisAlreadyEnded();

    genesisEnded = true;
    hburn.endGenesisMinting();

    emit GenesisPhaseEnded(totalTitanXBurned, totalHBURNMinted);

    if (genesisFundTitanX == 0 || lpReserveHBURN == 0) return;

    _createLiquidityPool(minWETHOut);
    // ❌ minWETHOut is entirely caller-controlled, no minimum floor
}

// GenesisBurn.sol; _createLiquidityPool() lines 359-375
function _createLiquidityPool(uint256 minWETHOut) internal {
    uint256 titanXBalance = genesisFundTitanX;
    IERC20(address(titanX)).forceApprove(address(swapRouter), titanXBalance);

    uint256 wethReceived = swapRouter.exactInputSingle(
        ISwapRouter.ExactInputSingleParams({
            tokenIn: address(titanX),
            tokenOut: address(weth),
            fee: titanXWethPoolFee,
            recipient: address(this),
            deadline: block.timestamp + 300,
            amountIn: titanXBalance,
            amountOutMinimum: minWETHOut,  // ❌ can be 0
            sqrtPriceLimitX96: 0
        })
    );
    // ...
}
```

Impact:

- MEV extraction from the Genesis Fund swap, reducing the WETH available for the permanant LP position
- The loss is one-time but irreversible since the LP-NFT is locked forever (whatever WETH makes it into the pool is all the protocol will ever have)
- A well-timed sandwich attack on a large genesis fund could extract a meaningful percentage of the swap value
- The damaged LP launches with less depth than intended, resulting in higher slippage for all future HBURN traders
- Even a well-intentioned caller might pass 0 out of convenience or misunderstanding, since the function doesn't guide them toward safe usage

FINDINGS

Medium Risk

M-01: No Enforcement of Reasonable Slippage on endGenesis LP Creation Swap (2)

Likelihood: Medium. The genesis fund swap is a one-time, high-value, publicly observable event. MEV bots on Ethereum mainnet routinely target exactly this type of unprotected swap. The only mitigating factor is that the caller *can* set a reasonable minWETHOut if they know to do so; but the contract does nothing to encourage or enforce this.

Simulation Steps:

1. Genesis concludes with genesisFundTitanX = 50,000 * 1e18 (50K TitanX accumulated from 8% of all burns).
2. A user (or bot) calls endGenesis(0) with zero slippage protection.
3. An MEV bot detects the pending transaction in the mempool.
4. The bot front-runs with a large TitanX buy on the TitanX/WETH pool, pushing the WETH price of TitanX down.
5. The genesis swap executes at the inflated rate, receiving significanlty less WETH than fair market value.
6. The bot back-runs by selling its TitanX position, capturing the spread.
7. The LP is created with less WETH than it should have, permanently weakening protocol liquidity.

Status: Open; recommended to remediate before deployment.

Category: MEV Exposure / Missing Validation

Recommendation:

- At minimum, enforce minWETHOut > 0 to prevent completely unprotected swaps, consistent with the BuyAndBurn contract's approach.
- For stronger protection, consider integrating an on-chain TWAP oracle from the TitanX/WETH Uniswap pool to compute a floor automatically. The contract could calculate the expected output based on a 30-minute TWAP and enforce that minWETHOut is within a reasonable tolerance (e.g., 95% of the TWAP-derived value).
- Alternatively, restrict endGenesis to a trusted guardian or multisig address that can be relied upon to supply an appropriate off-chain quote, rather than leaving it fully permissionless. This trades some decentralization for practical safety on a one-time critical operation.

FINDINGS

Medium Risk

M-02: block.timestamp Deadlines Provide No Actual Transaction Expiry Protection (1)

Component: GenesisBurn / _createLiquidityPool; BuyAndBurn / executeBuyAndBurn

Description: Both the GenesisBurn and BuyAndBurn contracts set swap deadlines as block.timestamp + 300 when calling the Uniswap V3 router. This pattern looks like it provides a 5-minute expiry window, but it actually provides no protection whatsoever. The block.timestamp value is determined by the validator at the time the transaction is included in a block, not at the time the transaction was submitted. A transaction sitting in the mempool for hours (or even days) will still satisfy the condition block.timestamp + 300 > block.timestamp the moment it finally gets included, regardless of how long it waited. This is a well-documented anti-pattern in DeFi security. The deadline parameter exists specifically to protect against stale transactions executing at outdated prices; setting it relative to block.timestamp defeats this purpose entirely. The same issue appears in both contracts, affecting the genesis fund TitanX-to-WETH swap and every BuyAndBurn WETH-to-HBURN swap.

```
// GenesisBurn.sol; _createLiquidityPool() lines 364-375
uint256 wethReceived = swapRouter.exactInputSingle(
    ISwapRouter.ExactInputSingleParams({
        tokenIn: address(titanX),
        tokenOut: address(weth),
        fee: titanXWethPoolFee,
        recipient: address(this),
        deadline: block.timestamp + 300,  // ❌ always passes, no matter when mined
        amountIn: titanXBalance,
        amountOutMinimum: minWETHOut,
        sqrtPriceLimitX96: 0
    })
);

// BuyAndBurn.sol; executeBuyAndBurn() lines 79-90
uint256 hburnBought = swapRouter.exactInputSingle(
    ISwapRouter.ExactInputSingleParams({
        tokenIn: address(weth),
        tokenOut: address(hburn),
        fee: POOL_FEE,
        recipient: address(this),
        deadline: block.timestamp + 300,  // ❌ same issue
        amountIn: ethBalance,
        amountOutMinimum: minHBURNOut,
        sqrtPriceLimitX96: 0
    })
);
```

Impact:

- Transactions can be held in the mempool by validators or through network congestion and executed at a much later time when market conditions have shifted unfavorably
- A validator could intentionally delay including a swap transaction, wait for the price to move against the swapper, and then include it alongside their own arbitrage trades
- For the genesis fund swap specifically (a one-time, high-value operation), a delayed execution could result in substantially worse pricing that permanantly affects the LP depth
- For BuyAndBurn, repeated execution at stale prices gradually erodes the effectiveness of the deflationary mechanism over time
- The minAmountOut parameter provides some protection against price deviation, but it does not protect against the scenario where the transaction is deliberately held and the market moves to exactly the minAmountOut threshold (the attacker profits from the spread between fair price and minimum)

Medium Risk

M-02: block.timestamp Deadlines Provide No Actual Transaction Expiry Protection (2)

Likelihood: Medium. On Ethereum mainnet, validators have full control over transaction ordering and inclusion timing. MEV-aware validators (which constitute a significant portion of Ethereum's validator set through services like Flashbots) routinely exploit stale transactions. The BuyAndBurn contract is particularly exposed since it will be called repeatedly over the protocol's lifetime, giving attackers many opportunities.

Simulation Steps:

1. A user calls executeBuyAndBurn(minHBURNOut) with a reasonable minimum output based on the current HBURN price.
2. The transaction enters the mempool. A validator sees the pending swap.
3. The validator holds the transaction without including it. Over the next several hours, the HBURN price drops significatly due to market activity.
4. The price drops to just above the minHBURNOut threshold. The validator now includes the transaction.
5. The swap executes and receives the minimum acceptable output rather than the fair market output at the time of submission. The validator (or a colluding searcher) captures the difference through arbitrage.
6. With block.timestamp + 300 as the deadline, no staleness check prevents this; the deadline is satisfied regardless of how long the transaction was delayed.

Status: Open; recommended to remediate before deployment.

Recommendation:

- Accept a caller-supplied deadline parameter instead of computing it on-chain. The deadline should be set off-chain relative to real wall-clock time at the moment of transaction submission.
- Apply the same pattern to endGenesis() by adding a deadline parameter that flows through to _createLiquidityPool.
- The frontend should set deadline to Math.floor(Date.now() / 1000) + 300 (current Unix time plus 5 minutes) so that the transaction genuinely expires if not mined within the intended window.

FINDINGS

Medium Risk

M-03: Penalty-Retained HBURN Accumulates in Staking Contract With No Distribution Path (1)

Component: HellBurnStaking / endStake

Description: When a user ends their stake between 50% and 100% maturity, a penalty is calculated and split into two equal portions. The first half is correctly burned by transferring to the dead address. The second half is described in comments as "stays in contract" and supposedly "benefits all holders via deflation." In reality though, this retained HBURN just sits in the contract's token balance with absolutely no mechanism to ever move it anywhere. It is not redistributed to active stakers, it is not added to any reward pool, it is not burned at a later point, and there is no admin function to recover it. The contract's endStake function returns HBURN to users based on s.amount (minus penalty), and _pendingETH only handles ETH rewards; neither path touches the accumulated penalty HBURN. The comment claims deflation benefit, but since these tokens are never actually burned (removed from totalSupply), they don't create real deflation either. They effectively become dead weight sitting in the contract, inflating the contract's HBURN balance in a way that could confuse off-chain analytics or portfolio trackers that interpret contract balances as meaningful.

```
// HellBurnStaking.sol; endStake() lines 162-183
uint256 penalty = 0;
uint256 amountReturned = s.amount;
uint256 maturityPct = _maturityPercent(s);

if (maturityPct < 50) {
    revert StakeNotMature();
} else if (maturityPct < 100) {
    uint256 penaltyPct = (100 - maturityPct) * 2;
    penalty = (s.amount * penaltyPct) / 100;
    amountReturned = s.amount - penalty;

    if (penalty > 0) {
        // 50% burned permanently, 50% stays in contract
        uint256 burnPortion = (penalty * PENALTY_BURN_PERCENT) / 100;
        uint256 keptPortion = penalty - burnPortion;

        hburn.safeTransfer(DEAD_ADDRESS, burnPortion);
        // keptPortion stays in contract (not transferred anywhere)
        // ❌ no accounting, no redistribution, no future burn
        // ❌ just orphaned HBURN growing over time

        emit PenaltyDistributed(burnPortion, keptPortion);
    }
}
```

Impact:

- HBURN accumulates permanently in the staking contract with no utility and no recovery mechanism
- The "deflation benefit" described in comments does not actually occur since totalSupply() is unaffected (tokens are held, not burned)
- Over time, the orphaned balance could grow to a non-trivial amount if early unstaking is common, especially during volatile market periods when users are more likely to exit early
- Off-chain dashboards or DeFi aggregators querying the contract's HBURN balance may misinterpret it as staked funds or backing collateral, leading to inaccurate protocol metrics
- If the protocol ever considers migration or upgrades through a new staking contract, this trapped HBURN cannot be carried forward

FINDINGS

Medium Risk

M-03: Penalty-Retained HBURN Accumulates in Staking Contract With No Distribution Path (2)

Likelihood: Medium. Early unstaking between 50-100% maturity is a normal user behavior that will occur regularly across the protocol's lifetime. Each early exit contributes to the orphaned balance. The financial loss per individual event is limited (its only half the penalty), but the cumulative effect over months or years of protocol operation is potentially significent.

Simulation Steps:

1. Alice stakes 100,000 HBURN for 100 days.
2. At day 75 (75% maturity), Alice decides to exit early. Penalty = (100 - 75) * 2 = 50% of her principal = 50,000 HBURN.
3. burnPortion = 25,000 HBURN (sent to dead address, actually removed from circulation in practice but not from totalSupply).
4. keptPortion = 25,000 HBURN (remains in the staking contract).
5. Alice receives 50,000 HBURN back. The transaction completes successfully.
6. The 25,000 HBURN sitting in the contract is now orphaned. No function exists to distribute, burn, or withdraw it.
7. Bob stakes and unstakes similarly. Another 10,000 HBURN gets trapped.
8. After dozens of early exits, the contract holds tens of thousands of HBURN that serve no purpose.
9. hburn.balanceOf(stakingContract) shows a balance that includes both legitimately staked tokens and orphaned penalty tokens, with no way to distinguish between the two on-chain.

Recommendation:

- The simplest and cleanest fix is to burn 100% of the penalty rather than splitting it. This creates actual deflation by reducing the dead-address-adjusted circulating supply, which is what the comment intended in the first place.
- If the team prefers to retain the 50/50 split for economic reasons, implement a redistribution mechanism. One approach is to track the kept portion in a penaltyPool variable and distribute it proportionally to active stakers (similar to how ethPerShare works but for HBURN), adding a secondary reward layer.
- A third option is to route the kept portion to the BuyAndBurn contract. The HBURN would be sold for ETH which then buys more HBURN on the open market, creating genuine buy pressure rather than dead accumulation. This is a bit circular but at least generates trading activity and LP fees.

FINDINGS

Medium Risk

M-04: finalizeUpTo Batch Finalization Reverts Entirely if a Single ETH Transfer Fails (1)

Component: BurnEpochs / finalizeUpTo, finalizeEpoch

Description: The finalizeUpTo function is a convenience method that lets anyone finalize multiple completed epochs in a single transaction, iterating from nextEpochToFinalize up to the specified epochId. During each iteration, if the epoch had burns and ETH deposits, 20% of the epoch's ETH is sent to the BuyAndBurn contract via a low-level .call{value: ...}. The problem is that if any single ETH transfer within the loop fails (for example the BuyAndBurn contract reverts on receive, runs out of gas, or is temporarily unable to accept ETH for whatever reason), the entire batch transaction reverts. This doesn't just fail the problematic epoch; it rolls back all state changes for every epoch in the batch, including epochs that were processed successfully before the failure. Since nextEpochToFinalize is a sequential pointer, you can't skip the problematic epoch either. The single-epoch finalizeEpoch function has the exact same issue but with less blast radius since it only processes one epoch at a time. However if the BuyAndBurn contract is persistently unable to receive ETH (say due to a bug or if its receive function exceeds the 2300 gas stipend for some reason), then even single-epoch finalization gets permanently stuck, blocking all future epoch settlements and locking all participant rewards.

```
// BurnEpochs.sol; finalizeUpTo() lines 181-211
function finalizeUpTo(uint256 epochId) external nonReentrant {
    uint256 start = nextEpochToFinalize;
    for (uint256 i = start; i <= epochId; i++) {
        Epoch storage epoch = epochs[i];
        if (!_isEpochEnded(i)) break;
        if (epoch.finalized) continue;

        nextEpochToFinalize = i + 1;

        uint256 epochETH = epoch.ethDeposited + carryOverETH;
        carryOverETH = 0;
        epoch.finalized = true;

        if (epochETH > 0 && epoch.totalWeightedBurns > 0) {
            uint256 rewardsForEpoch = (epochETH * REWARDS_PERCENT) / 100;
            uint256 buyBurnAmount = epochETH - rewardsForEpoch;
            epoch.ethRewards = rewardsForEpoch;
            totalETHDistributed += epoch.ethRewards;

            if (buyBurnAmount > 0) {
                (bool sent,) = buyAndBurn.call{value: buyBurnAmount}("");
                if (!sent) revert TransferFailed();
                // ❌ single failed transfer reverts ALL epochs in the batch
                // ❌ state changes for previously processed epochs are rolled back
            }
        } else if (epochETH > 0) {
            carryOverETH = epochETH;
            emit OrphanedETHCarriedOver(i, epochETH);
        }

        emit EpochFinalized(i, epoch.ethRewards, epoch.totalWeightedBurns);
    }
}

// BurnEpochs.sol; finalizeEpoch() lines 143-178
// Same pattern for single epoch
if (buyBurnAmount > 0) {
    (bool sent,) = buyAndBurn.call{value: buyBurnAmount}("");
    if (!sent) revert TransferFailed();
    // ❌ if BuyAndBurn persistently rejects ETH, finalization is permanently blocked
}
```

Impact:

- A single failing ETH transfer blocks finalization of all pending epochs, not just the one that failed
- In the batch case, epochs that were already processed within the same loop get rolled back, wasting gas and leaving the system in the same state as before the call
- If BuyAndBurn becomes permanantly unable to accept ETH (contract bug, self-destruct, gas limit issues), the entire epoch system halts with no workaround
- All participant rewards for every unfinalized epoch become inaccessible since claimRewards requires epoch.finalized == true
- The sequential nextEpochToFinalize pointer means there's no way to skip a problematic epoch and finalize later ones
- carryOverETH is zeroed mid-loop before the external call, so a revert could theoretically cause accounting issues in edge cases where the revert restores a stale carryOverETH state (though Solidity's atomic revert does handle this correctly, the pattern is still fragile)

FINDINGS

Medium Risk

M-04: finalizeUpTo Batch Finalization Reverts Entirely if a Single ETH Transfer Fails (2)

Likelihood: Medium. The BuyAndBurn contract currently has a simple receive() external payable {} that should always accept ETH. But "should always work" is not the same as a guarantee on mainnet. Contract upgrades through redeployment, interactions with MEV protection services, or even the BuyAndBurn contract being the target of a griefing attack (e.g. someone self-destructs a contract to it to cause unexpected behavior) could create conditions where the transfer fails. The risk is lower for the single-epoch path but the consequence is the same if BuyAndBurn is persistently broken.

Simulation Steps:

1. Ten epochs complete without being finalized. Each has ETH deposits and burn participants waiting for rewards.
2. A keeper calls finalizeUpTo(9) to settle all ten at once.
3. Epochs 0 through 6 are processed successfully within the loop. State changes are made but not yet committed (still within the same transaction).
4. On epoch 7, the buyAndBurn.call{value: buyBurnAmount} fails because the BuyAndBurn contract temporarily cannot accept ETH.
5. The entire transaction reverts. All state changes from epochs 0 through 6 are rolled back.
6. nextEpochToFinalize is still 0. None of the ten epochs are finalized. No rewards are claimable.
7. The keeper tries finalizeEpoch(0) individually. If BuyAndBurn is still broken, even this single call reverts.
8. All participants across all ten epochs are locked out of their rewards until someone fixes or redeploys the BuyAndBurn contract (which is immutable, so redeployment isn't an option for the epoch system).

Status: Open; recommended to remediate before deployment.

Category: Denial of Service / External Call Dependency

Recommendation:

- Accumulate the total BuyAndBurn allocation across all epochs and perform a single transfer at the end of the loop. This limits the external call exposure to one point of failure and reduces gas costs.
- For stronger resilience, use a pull pattern instead of push. Accumulate the BuyAndBurn allocation in a pendingBuyBurnETH state variable and let the BuyAndBurn contract (or anyone) withdraw it separately. This fully decouples finalization from the external transfer
- This pull-based approach ensures that epoch finalization and reward distribution can never be blocked by an external contract's behavior.

FINDINGS

Medium Risk

M-05: HBURN Burned to Dead Address Instead of Using burn() ; totalSupply() Never Decreases (1)

Component: GenesisBurn / collectLPFees; BuyAndBurn / executeBuyAndBurn; HellBurnStaking / endStake; BurnEpochs / _burn

Description: Across the entire protocol, every instance of "burning" HBURN is implemented by transferring tokens to 0x000000000000000000000000000000000000dEaD rather than calling the burn() function inherited from OpenZeppelin's ERC20Burnable. The HellBurnToken contract explicitly inherits ERC20Burnable, making the burn() and burnFrom() functions available, but they are never used by any of the protocol's contracts. The practical consequence is that totalSupply() never decreases regardless of how much HBURN the protocol removes from circulation. The dead address simply accumulates an ever-growing balance. This matters because totalSupply() is the standard method used by DeFi protocols, analytics platforms, price aggregators, DEX interfaces, and portfolio trackers to determine a token's circulating supply and by extension its market capitalization. A protocol that markets itself as "deflationary" but whose totalSupply() remains constant (or only decreases through user-initiated burns) creates a disconnect between the actual economic reality and what third-party systems report. CoinGecko, CoinMarketCap, Etherscan's token tracker, and Uniswap's own analytics all rely on totalSupply() for their calculations. An overstated supply leads to an understated price-per-token in market cap calculations, which can deter potential participants who evaluate projects based on these widely-used metrics.

```
// BuyAndBurn.sol; executeBuyAndBurn() lines 92-94
// The protocol's primary deflationary engine sends to dead address
uint256 burnAmount = hburn.balanceOf(address(this));
hburn.safeTransfer(DEAD_ADDRESS, burnAmount);
// ❌ totalSupply() unchanged; tokens still "exist" at the dead address

// GenesisBurn.sol; collectLPFees() lines 289-291
// LP fee HBURN also sent to dead address
if (hburnFees > 0) {
    IERC20(address(hburn)).safeTransfer(DEAD_ADDRESS, hburnFees);
    // ❌ same pattern
}

// HellBurnStaking.sol; endStake() lines 178-179
// Early unstake penalty burn
uint256 burnPortion = (penalty * PENALTY_BURN_PERCENT) / 100;
hburn.safeTransfer(DEAD_ADDRESS, burnPortion);
// ❌ same pattern again

// Meanwhile, the actual burn functionality sits unused:
// HellBurnToken.sol inherits ERC20Burnable
contract HellBurnToken is ERC20, ERC20Burnable, ReentrancyGuard {
    // burn() and burnFrom() are available but never called by protocol contracts
}
```

Impact:

- totalSupply() remains static after genesis ends, regardless of how much HBURN is removed from circuation through BuyAndBurn, LP fee collection, or staking penalties
- Market cap calculations on every major analytics platform will be overstated since they multiply price by totalSupply()
- DeFi protocols that integrate HBURN and use totalSupply() for any on-chain logic (ratio calculations, collateral valuations, governance weight) will operate on incorrect data
- The protocol's core value proposition is deflation, but the most standard and universally queried metric for measuring deflation (totalSupply()) shows no deflation occurring
- Dead address accumulation is technically verifiable but requires custom queries that most users and platforms won't perform; the default experience for anyone checking the token is a static supply

FINDINGS

Medium Risk

M-05: HBURN Burned to Dead Address Instead of Using burn() ; totalSupply() Never Decreases (2)

Likelihood: Medium. This isn't a direct exploit but rather a persistent misrepresentation that affects every external system interacting with the token from day one. The impact compounds over time as the BuyAndBurn contract removes more and more HBURN from circulation without totalSupply() reflecting any of it. For a protocol where deflation is the central thesis, having the primary deflation metric broken is a meaningful problem.

Simulation Steps:

1. Genesis ends with totalSupply() = 920,000,000,000 HBURN (hypothetical example).
2. Over the following months, BuyAndBurn purchases and "burns" 50 billion HBURN by sending it to the dead address.
3. LP fee collection "burns" another 5 billion HBURN to the dead address.
4. Staking penalties "burn" 2 billion HBURN to the dead address.
5. hburn.totalSupply() still returns 920,000,000,000. Zero change.
6. hburn.balanceOf(0x...dEaD) returns 57,000,000,000 but nobody checks this by default.
7. CoinGecko shows market cap as price * 920B instead of the economically accurate price * 863B.
8. A potential investor comparing HBURN's market cap to competitors sees an inflated number and draws incorrect conclusions about valuation.

Status: Open; recommended to remediate before deployment.

Category: Token Accounting / External Integration

Recommendation:

- Replace all safeTransfer(DEAD_ADDRESS, ...) patterns with actual burn() calls. Since the protocol contracts hold the tokens before burning them, they can call burn() directly without needing burnFrom approvals
- If the team specifically wants dead-address transparency (some communities prefer seeing the dead address balance as a visible burn tracker), consider implementing a hybrid approach: call burn() for actual supply reduction but also maintain a public totalHBURNBurned counter in each contract so the cumulative burn history is easily queryable without relying on the dead address balance.
- At the very least, add a circulatingSupply() view function to HellBurnToken that subtracts the dead address balance from totalSupply(), giving integrators an accurate number to work with

FINDINGS

Low Risk

L-01: ETH Deposit Allocated to Wrong Epoch at Boundary Timestamp (1)

Component: BurnEpochs / _recordETHDeposit, currentEpochId, receive

Description: When ETH is received by the BurnEpochs contract, the _recordETHDeposit function determines which epoch to credit by calling currentEpochId(). This function computes the epoch index using integer division: (block.timestamp - firstEpochStart) / EPOCH_DURATION. At the exact moment an epoch boundary occurs (i.e. block.timestamp equals exactly the start of a new epoch), the integer division rolls over and the deposit gets credited to the new epoch rather than the one that just ended. In most cases this is the intended behavior since the new epoch has technically started. The issue is more subtle than that though. External protocols, automation bots, or even the finalizeEpoch function itself may be sending ETH in the same block or same transaction batch as the epoch transition. ETH that was "earned" during the previous epoch (for instance from staking payouts or protocol revenue that accumulated over the prior 8 days) could end up credited to the brand new epoch that has zero participants yet. If the new epoch ends up with no burners, that ETH becomes carry-over rather than going to the participants who actually generated it. The timing window is narrow (a single block at the boundary) but on Ethereum where block builders control transaction ordering, its not impossible for this to be exploited or to occur accidentally with MEV-aware infrastructure.

```
// BurnEpochs.sol; _recordETHDeposit() lines 306-311
function _recordETHDeposit(uint256 amount) internal {
    if (amount == 0) return;
    uint256 epochId = currentEpochId();
    // ❌ at exact boundary, this returns the NEW epoch
    epochs[epochId].ethDeposited += amount;
    emit ETHReceived(epochId, amount);
}

// BurnEpochs.sol; currentEpochId() lines 258-261
function currentEpochId() public view returns (uint256) {
    if (block.timestamp < firstEpochStart) return 0;
    return (block.timestamp - firstEpochStart) / EPOCH_DURATION;
    // at boundary: (firstEpochStart + N * EPOCH_DURATION - firstEpochStart)
    // EPOCH_DURATION = N
    // rolls to new epoch exactly at the transition second
}
```

Impact:

- ETH that logically belongs to the previous epoch's reward pool could be misallocated to the next epoch
- If the next epoch has no participants, the misallocated ETH becomes carry-over and gets delayed by an additional epoch cycle (8 days minimum)
- In the worst case, if multiple consecutive epochs have no participants, the ETH keeps rolling over indefinitely
- The amount at risk per occurrence is limited to whatever ETH arrives in the exact boundary block, so individual impact is small
- However over many epoch transitions across the protocol's lifetime, the cumulative misallocation could become noticeable

FINDINGS

Low Risk

L-01: ETH Deposit Allocated to Wrong Epoch at Boundary Timestamp (2)

Likelihood: Low. The vulnerable window is exactly one block at each epoch boundary (every 8 days). Natural ETH deposits from human users are unlikely to hit this exact moment. Automated systems or contracts that send ETH on a schedule are more likely candidates but the probability is still low per-occurrence.

Status: Open; minor issue, recommended to address if practical.

Category: Timing / Edge Case Accounting

Recommendation:

- Consider adding a small buffer or grace period to _recordETHDeposit that checks whether the current epoch has just started (within the last few seconds) and whether the ETH might logically belong to the prior epoch. Alternatively, document this as a known edge case and accept the minor misallocation risk.
- Another approach is to allow finalizeEpoch to snapshot the contract's ETH balance at finalization time rather than relying purely on incremental deposit tracking, though this would require rethinking the per-epoch accounting model.
- At minimum, ensure any off-chain automation or keeper bots that send ETH to the contract are configured to avoid sending within a few blocks of the epoch boundary.

FINDINGS

Low Risk

L-02: batchClaimRewards Silently Skips Invalid Epochs With No Feedback (1)

Component: BurnEpochs / batchClaimRewards

Description: The batchClaimRewards function accepts an array of epoch IDs and iterates through them, claiming rewards for each one where the caller is eligible. If an epoch is not finalized, already claimed, or the user had zero burns in it, the function simply continues past it without any indication that the epoch was skipped. The transaction succeeds even if every single epoch in the array was skipped and zero ETH was distributed. There's no revert, no event, and no return value telling the caller how many epochs were actually processed versus skipped. Compare this to the single-epoch claimRewards function which correctly reverts with specific errors (EpochNotFinalized, AlreadyClaimed, NoBurnsInEpoch) when the caller is not eligible. The batch version swallows all of these conditions silently. From a user's perspective, they submit a transaction, it succeeds, gas is consumed, and they assume their rewards were claimed. They might not notice for days or weeks that their balance didn't actually change, especially if they're claiming across many epochs and not carefully tracking the expected amount for each one.

```
// BurnEpochs.sol; batchClaimRewards() lines 233-255
function batchClaimRewards(uint256[] calldata epochIds) external nonReentrant {
    uint256 totalReward = 0;

    for (uint256 i = 0; i < epochIds.length; i++) {
        Epoch storage epoch = epochs[epochIds[i]];
        if (!epoch.finalized) continue;          // ❌ silent skip
        if (epoch.claimed[msg.sender]) continue;  // ❌ silent skip
        if (epoch.userWeightedBurns[msg.sender] == 0) continue;  // ❌ silent skip

        uint256 userShare = (epoch.ethRewards * epoch.userWeightedBurns[msg.sender])
                            / epoch.totalWeightedBurns;

        epoch.claimed[msg.sender] = true;
        totalReward += userShare;

        emit RewardsClaimed(msg.sender, epochIds[i], userShare);
    }

    if (totalReward > 0) {
        (bool sent,) = msg.sender.call{value: totalReward}("");
        if (!sent) revert TransferFailed();
    }
    // ❌ no revert when totalReward == 0
    // ❌ no return value indicating how many epochs were processed
    // transaction just succeeds silently with nothing happening
}
```

Impact:

- Users receive no feedback when their batch claim processes zero epochs, leading to a false sense of having collected rewards
- Gas is wasted on transactions that accomplish nothing, which could be frustrating for users on a tight budget who don't realize they need to wait for finalization first
- Frontend applications that rely on transaction success as a signal that rewards were claimed will incorrectly update their UI state
- Not a direct financial loss since no funds are at risk, but it degrades the user experience and could erode trust if people feel the protocol "ate" their rewards

FINDINGS

Low Risk

L-02: batchClaimRewards Silently Skips Invalid Epochs With No Feedback (2)

Likelihood: Low. This requires the user to pass epoch IDs that are all either unfinalized, already claimed, or epochs they didn't participate in. Likely to happen when users are batch-claiming speculatively (passing a range of epoch IDs without checking eligibility first) or when a frontend has stale data about which epochs are finalized.

Status: Open; minor improvement recommended.

Category: User Experience / Silent Failure

Recommendation:

- Add a revert when totalReward == 0 at the end of the function, similar to how claimVested in GenesisBurn reverts with NothingToClaim. This gives the user immediate feedback that none of their specified epochs yielded rewards.
- Optionally, return the number of epochs successfully processed and the total ETH claimed, allowing frontends to display accurate post-transaction information without needing to parse event logs.
- If the team prefers keeping the silent-skip behavior for composability reasons (some batch callers might not want reverts on partial failure), consider emitting a summary event at the end of the function that includes the total epochs attempted, epochs successfully claimed, and total ETH distributed. This at least gives off-chain systems something to work with.

FINDINGS

Low Risk

L-03: Dust-Level Fuel Burns Still Increment Shares Due to Minimum Floor of 1 (1)

Component: HellBurnStaking / _addFuel

Description: When a user adds fuel to their stake, the fuel increment is calculated as (amount * weight) / 1e9. If the result rounds down to zero (which happens for any TitanX amount below 1e9 wei, roughly 0.000000001 TitanX), the code sets fuelIncrement = 1 as a floor. This means a user can burn literally 1 wei of TitanX and still receive a fuel bonus increment of 1, which triggers a full share recalculation. On its own this sounds harmless since a fuel increment of 1 on a basis of 1000 is negligible. But the real problem is that this interacts with C-01 (the missing ETH debt reset in _addFuel). Every fuel addition, no matter how small, recalculates shares without updating stakeETHDebt, retroactively inflating the user's pending ETH rewards. The dust floor makes this exploit essentially free to repeat; the attacker burns 1 wei of TitanX each time, pays only gas, and extracts a small amount of other stakers' ETH on every call. Over many iterations (which could be batched through a contract), the cumulative theft adds up. Even without C-01, the floor creates a slight economic inefficiency where users get non-zero fuel credit for economically meaningless burns, though the impact in isolation is truly negligible.

```
// HellBurnStaking.sol; _addFuel() lines 296-301
// Calculate fuel increment
uint256 fuelIncrement = (amount * weight) / 1e9;
if (fuelIncrement == 0) fuelIncrement = 1;
// ❌ 1 wei of TitanX still yields fuelIncrement = 1
// ❌ triggers full share recalculation below

// Remove old shares, recalculate with new fuel
totalShares -= s.shares;
s.fuelBonus = s.fuelBonus + fuelIncrement;
if (s.fuelBonus > MAX_FUEL_BONUS) s.fuelBonus = MAX_FUEL_BONUS;

uint256 numDays = (s.endTime - s.startTime) / 1 days;
uint256 timeBonus = _timeBonus(numDays);
s.shares = (s.amount * timeBonus * s.fuelBonus * s.loyaltyBonus) / (BASIS * FUEL_BASIS * BASIS);

totalShares += s.shares;
// full recalculation triggered by a 1 wei burn
```

Impact:

- In combination with C-01, this makes the retroactive ETH theft exploit essentially free to execute repeatedly (cost is just gas plus 1 wei of TitanX per call)
- An attacker could write a contract that loops addFuelTitanX(stakeId, 1) dozens of times in a single transaction, incrementally inflating their shares and draining other stakers' rewards at minimal cost
- The fuel bonus cap at 1500 does limit how many times this can be exploited before FuelMaxReached reverts, but reaching the cap from the default 1000 takes 500 dust-level calls, each one triggering the C-01 exploit along the way
- In isolation (without C-01), the impact is negligible since the share inflation per call is astronomically small

FINDINGS

Low Risk

L-03: Dust-Level Fuel Burns Still Increment Shares Due to Minimum Floor of 1 (2)

Likelihood: Low in isolation. High when combined with C-01 since it removes the economic barrier to repeated exploitation. If C-01 is fixed, this finding drops to purely informational.

Status: Open; minor issue that becomes significant in combination with C-01.

Category: Input Validation / Economic Inefficiency

Recommendation:

- Enforce a meaningful minimum amount on fuel burns rather than using a floor of 1. A reasonable minimum would be something like 1e18 (1 full TitanX token) to ensure that every fuel addition represents a real economic commitment.
- Remove the `if (fuelIncrement == 0) fuelIncrement = 1` floor entirely. If the calculated increment rounds to zero, the burn amount was too small to matter and the function should revert instead of granting phantom credit.
- Even after C-01 is fixed, removing the dust floor is good hygiene. It prevents users from wasting gas on meaningless fuel additions and keeps the fuel mechanic economically coherent.

FINDINGS

Low Risk

L-04: No Validation That firstEpochStart Is in the Future (1)

Component: BurnEpochs / constructor

Description: The BurnEpochs constructor accepts _firstEpochStart as a raw uint256 timestamp with no validation that it falls in the future relative to block.timestamp. If the deployer accidentally passes a timestamp in the past (a typo, a stale value from a test environment, copy-pasting from a previous deployment, or simply forgetting to update it), the contract will deploy successfully and immediately consider one or more epochs as already elapsed. The currentEpochId() function would return a non-zero value from the very first block, and nextEpochToFinalize would be out of sync since it starts at 0 but the "current" epoch might already be 5 or 10 depending on how far in the past the start time was set. Users calling burnTitanX or burnDragonX would be participating in an epoch that's technically already over (or close to ending), and finalizing the skipped epochs would need to happen sequentially from 0 through each empty epoch before reaching the actually active one. None of this causes direct fund loss, but it creates a confusing launch state where the protocol appears broken, epochs need to be finalized in bulk before anything works properly, and early users might burn tokens into epochs that get finalized with zero ETH rewards because nobody had time to deposit.

```
// BurnEpochs.sol; constructor lines 97-115
constructor(
    address _titanX,
    address _dragonX,
    address _buyAndBurn,
    address _treasury,
    uint256 _firstEpochStart,   // ❌ no validation against block.timestamp
    address _guardian
) {
    require(_titanX != address(0) && _dragonX != address(0), "zero addr");
    require(_buyAndBurn != address(0) && _treasury != address(0), "zero addr");
    require(_guardian != address(0), "zero addr");

    titanX = ITitanX(_titanX);
    dragonX = _dragonX;
    buyAndBurn = _buyAndBurn;
    treasury = _treasury;
    guardian = _guardian;
    firstEpochStart = _firstEpochStart;
    // ❌ no require(_firstEpochStart >= block.timestamp)
}

// What happens when firstEpochStart is in the past:
// currentEpochId() lines 258-261
function currentEpochId() public view returns (uint256) {
    if (block.timestamp < firstEpochStart) return 0;
    return (block.timestamp - firstEpochStart) / EPOCH_DURATION;
    // if firstEpochStart was 30 days ago: (30 days) / (8 days) = epoch 3
    // epochs 0, 1, 2 are "ended" but never had participants
}
```

Impact:

- A misconfigured deployment creates phantom elapsed epochs that need to be finalized sequentially before the protocol reaches a normal operating state
- Early users may burn tokens into an epoch that's about to end (or already ended), receiving no rewards for their participation
- The nextEpochToFinalize pointer starts at 0 regardless, so someone would need to call finalizeUpTo across all the phantom epochs before reaching the live one; each empty epoch triggers the carry-over path, which works correctly but wastes gas and creates confusing events
- No direct fund loss occurs since empty epochs simply carry over their ETH, but the user experience at launch would be poor and could damage confidence in the protocol

FINDINGS

Low Risk

L-04: No Validation That firstEpochStart Is in the Future (2)

Likelihood: Low. This is purely a deployment misconfiguration risk. A competent deployment process with proper parameter verification would catch this. The test suite does set firstEpochStart in the future (current time plus one day), suggesting the team is aware of the intended usage. But constructor-level validation is cheap insurance against human error during a high-pressure mainnet deployment.

Status: Open; minor issue, recommended to add as a defensive measure.

Category: Deployment Safety / Input Validation

Recommendation:

- Add a simple require in the constructor that enforces _firstEpochStart >= block.timestamp. This costs negligible deployment gas and eliminates an entire class of misconfiguration errors.
- If the team wants flexibility to deploy the contract slightly ahead of time and have it "go live" at a specific future block, they could use a softer check like require(_firstEpochStart >= block.timestamp - 1 hours) to allow a small window for deployment delays while still catching obviously wrong values like timestamps from last year.

FINDINGS

Low Risk

L-05: Immutable Companion Contract Addresses Prevent Recovery if Any Contract Needs Redeployment (1)

Component: HellBurnToken / constructor

Description: The HellBurnToken contract stores three critical addresses as immutables: genesisContract, stakingContract, and buyAndBurnContract. These are set once during construction and can never be changed. The genesisContract address is particularly important because it controls the only minting pathway; if the GenesisBurn contract has a bug discovered after deployment but before genesis ends, there is no way to point the token at a new genesis contract. Minting would be permanently broken or permanently exploitable depending on the nature of the bug. The same applies in the other direction; if HellBurnToken itself needs redeployment, every other contract in the system that references its address (GenesisBurn, HellBurnStaking, BuyAndBurn, BurnEpochs) would also need redeployment since they all store the HBURN address as an immutable too. This creates a rigid dependency web where a bug in any single contract effectively requires redeploying the entire protocol from scratch, including re-running the genesis phase. To be clear, immutability is generally a security feature not a weakness; it prevents admin key attacks and rug pulls, which aligns with the protocol's trustless design philosophy. But its worth understanding the tradeoff explicitly: the protocol gains trust guarantees at the cost of zero recoverability if something goes wrong post-deployment.

```
// HellBurnToken.sol; constructor and state lines 17-40
address public immutable genesisContract;
address public immutable stakingContract;
address public immutable buyAndBurnContract;

constructor(
    address _genesis,
    address _staking,
    address _buyBurn
) ERC20("HellBurn", "HBURN") {
    require(_genesis != address(0) && _staking != address(0) && _buyBurn != address(0), "zero addr");
    genesisContract = _genesis;    // ❌ permanently locked
    stakingContract = _staking;    // ❌ permanently locked
    buyAndBurnContract = _buyBurn; // ❌ permanently locked
}

// Minting is exclusively gated to the genesis address
function mint(address to, uint256 amount) external {
    if (msg.sender != genesisContract) revert OnlyGenesis();
    // no other address can ever mint, even if genesis contract is buggy
    if (genesisMintingEnded) revert MintingEnded();
    _mint(to, amount);
}

// Similarly, GenesisBurn stores HellBurnToken as immutable:
// GenesisBurn.sol line 70
HellBurnToken public immutable hburn;

// HellBurnStaking stores it as immutable:
// HellBurnStaking.sol line 45
IERC20 public immutable hburn;

// BuyAndBurn stores it as immutable:
// BuyAndBurn.sol line 29
IERC20 public immutable hburn;
// Every contract is permanently bound to every other contract
```

Impact:

- If a critical vulnerability is found in GenesisBurn during the live genesis phase, there is no way to pause minting and redirect to a patched contract; the only options are to let the exploit continue for up to 28 days or to abandon the deployment entirely and start over
- Users who already participated in genesis would need to be made whole through some off-chain process if a full redeployment occurs, which contradicts the trustless design goals
- The BuyAndBurn contract is particularly exposed since it operates indefinitely after genesis; a bug found months into operation would mean all accumulated ETH in the contract is stuck unless the Uniswap swap still works correctly
- This is an architectural tradeoff not a bug. The protocol deliberately chose maximum immutability over recoverability, which is a legitimate design choice that should be understood by all stakeholders

FINDINGS

Low Risk

L-05: Immutable Companion Contract Addresses Prevent Recovery if Any Contract Needs Redeployment (2)

Likelihood: Low. This only becomes a problem if a bug is discovered post-deployment, which this audit aims to prevent. The code is relatively compact (around 1,500 lines total) and the logic is straightforward enough that thorough auditing and testing can provide reasonable confidence. But "reasonable confidence" is not the same as certainty, and the consequences of being wrong are severe given the zero-recoverability architecture.

Status: Open; architectural observation, not a direct vulnerability.

Category: Architecture / Recoverability Tradeoff

Recommendation:

- This finding does not necessarily require a code change since the immutable architecture is a deliberate and defensible design decision. However the team should ensure that all stakeholders (investors, community, launch partners) understand the tradeoff explicitly: the protocol cannot be patched after deployment under any circumstances.
- Consider performing multiple independent audits before mainnet deployment given the zero-recoverability constraint. A single audit pass provides one perspective; two or three independent reviews dramatically reduce the probability of a missed vulnerability.
- For future protocol iterations, consider a time-locked governance pattern where companion addresses can be updated but only after a mandatory delay (e.g., 30 days). This preserves most of the trust guarantees while allowing emergency recovery in extreme scenarios.
- At minimum, ensure the deployment script is heavily tested and verified on a fork of mainnet before executing on mainnet itself. A misconfigured constructor parameter in this architecture is unrecoverable.

FINDINGS

Informational

I-01: Genesis Fund Amount Computed as Remainder Rather Than Direct Percentage In _distributeTitanX, the genesis fund amount is calculated as amount - burnAmount - dragonAmount - treasuryAmount instead of (amount * 8) / 100. This is actually good practice since it captures rounding dust that would otherwise be lost, but it means the genesis fund may not be exactly 8% on every call. The deviation is at most a few wei per transaction so it has no practical impact, but it's worth documenting for accounting transparency.

I-02: Test Suite Uses Mock Uniswap Contracts That Don't Replicate Real Pool Behavior The MockSwapRouter returns 1:1 swaps and the MockNonfungiblePositionManager doesn't enforce tick math, price bounds, or liquidity concentration rules. This means the LP creation flow, sqrtPriceX96 calculation, and fee collection logic have never been tested against actual Uniswap V3 behavior. Edge cases around token ordering, price initialization failures, or partial liquidity deposits could surface on mainnet that the test suite would never catch. Strongly recommended to add fork-based integration tests against real Uniswap V3 deployments on a mainnet fork before going live.

I-03: completedStakes Counter Increments on Early Exits, Not Just Fully Matured Stakes In HellBurnStaking.endStake(), the completedStakes[msg.sender]++ line executes regardless of whether the stake reached full maturity or was exited early at 50-80% with a penalty. This means a user who repeatedly stakes and exits early at 51% maturity accumulates "completed stakes" and becomes eligible for reStake and its loyalty bonuses. Depending on the team's intent, this might be undesirable since the loyalty system seems designed to reward committed long-term stakers, not users who bail early and re-enter. Worth clarifying the intended behavior and adjusting the condition if only fully matured stakes should count.

I-04: onERC721Received Accepts Any NFT, Not Just the Uniswap LP Position The GenesisBurn contract implements onERC721Received to hold the Uniswap V3 LP-NFT, but the function accepts any ERC-721 token from any sender without validation. Random NFTs accidentally (or intentionally) sent to the contract become permanently locked since there's no transfer-out function. This has no financial impact on the protocol itself but could result in third-party NFTs being irrecoverably lost. Consider validating that the sender is the NonfungiblePositionManager and that the tokenId matches the expected LP position.

CONCLUSION

The HellBurn protocol presents a well-structured and intentionally minimalist DeFi architecture built on a foundation of immutability and trustlessness. The codebase is compact, the inheritance patterns are clean, and the team has clearly invested effort in iterative hardening during development (the codebase contains annotations labeled "Audit Fixes v2" referencing specific issue IDs, suggesting prior internal or external review that we were not provided documentation for). The use of OpenZeppelin primitives for reentrancy protection, safe token transfers, and pausability reflects solid baseline security hygiene. The decision to eliminate all admin keys and owner functions is a strong trust signal that aligns with the protocol's positioning within the TitanX ecosystem.

That said, this review identified two Critical severity findings that require immediate remediation before any mainnet deployment. C-01 (the missing ETH debt reset in the fuel mechanic) is a direct fund-theft vector that allows any staker to drain ETH rewards from other participants at near-zero cost. C-02 (the unvalidated recipient in LP fee collection) allows anyone to redirect accumulated WETH trading fees to an arbitrary address. Both of these are straightforward to fix and the recommended remediations are non-invasive; they don't require architectural changes, just targeted additions to existing functions.

The three High severity findings represent meaningful risks that could result in permanent fund loss under realistic conditions. H-01 (ETH lost when received with zero stakers) is a gap that the BurnEpochs contract handles correctly through its carry-over mechanism but the staking contract does not. H-02 (precision loss in the LP price calculation) is a one-time risk at genesis conclusion but the consequences are irreversible since the LP position is permanently locked. H-03 (unbounded vesting array) is primarily a self-griefing vector but could trap legitimate funds if a user interacts with genesis through an automated strategy that creates many small burns.

The Medium findings center around two themes. The first is MEV exposure; both the genesis fund swap and the ongoing BuyAndBurn swaps lack robust slippage and deadline protections, leaving value on the table for sandwich attackers. The second is token accounting; the protocol's reliance on dead-address transfers instead of actual burn() calls means the deflationary narrative is not reflected in totalSupply(), which is the single most queried metric by external platforms and integrators.

The Low and Informational findings are minor edge cases and architectural observations that do not pose immediate risk but would improve robustness and code quality if addressed.

It is worth noting that the protocol's fully immutable architecture (documented in L-05) means that none of these findings can be patched after deployment. Every issue must be resolved in the source code and verified through testing before the contracts go to mainnet. There is no upgrade path, no proxy pattern, and no governance mechanism that could apply a post-deployment fix. This makes the pre-deployment audit and remediation cycle especially critical for HellBurn compared to protocols that retain some form of upgradeability.

Overall, the protocol's design philosophy is sound and the codebase is above average in quality for projects in the TitanX ecosystem. The identified issues are fixable without fundamental redesign. We strongly recommend the team address all Critical and High findings, carefully consider the Medium findings (particularly around MEV protection and token accounting), and submit the remediated code for a focused follow-up review before proceeding to mainnet deployment.

SPYWOLF

CRYPTO SECURITY

Audits | KYCs | dApps
Contract Development

ABOUT US

We are a growing crypto security agency offering audits, KYCs and consulting services for some of the top names in the crypto industry.

- ✔ **OVER 700 SUCCESSFUL CLIENTS**
- ✔ **MORE THAN 1000 SCAMS EXPOSED**
- ✔ **MILLIONS SAVED IN POTENTIAL FRAUD**
- ✔ **PARTNERSHIPS WITH TOP LAUNCHPADS, INFLUENCERS AND CRYPTO PROJECTS**
- ✔ **CONSTANTLY BUILDING TOOLS TO HELP INVESTORS DO BETTER RESEARCH**

To hire us, reach out to contact@spywolf.co or t.me/joe_SpyWolf

FIND US ONLINE

SPYWOLF.CO

@SPYWOLFNETWORK

@SPYWOLFNETWORK

Disclaimer

This report shows findings based on our limited project analysis, following good industry practice from the date of this report, in relation to cybersecurity vulnerabilities and issues in the framework and algorithms based on smart contracts, overall social media and website presence and team transparency details of which are set out in this report. In order to get a full view of our analysis, it is crucial for you to read the full report.
While we have done our best in conducting our analysis and producing this report, it is important to note that you should not rely on this report and cannot claim against us on the basis of what it says or doesn't say, or how we produced it, and it is important for you to conduct your own independent investigations before making any decisions. We go into more detail on this in the disclaimer below – please make sure to read it in full.

DISCLAIMER:
By reading this report or any part of it, you agree to the terms of this disclaimer. If you do not agree to the terms, then please immediately cease reading this report, and delete and destroy any and all copies of this report downloaded and/or printed by you. This report is provided for information purposes only and on a non-reliance basis, and does not constitute investment advice.
No one shall have any right to rely on the report or its contents, and SpyWolf and its affiliates (including holding companies, shareholders, subsidiaries, employees, directors, officers and other representatives) (SpyWolf) owe no duty of care towards you or any other person, nor does SpyWolf make any warranty or representation to any person on the accuracy or completeness of the report.
The report is provided "as is", without any conditions, warranties or other terms of any kind except as set out in this disclaimer, and SpyWolf hereby excludes all representations, warranties, conditions and other terms (including, without limitation, the warranties implied by law of satisfactory quality, fitness for purpose and the use of reasonable care and skill) which, but for this clause, might have effect in relation to the report. Except and only to the extent that it is prohibited by law, SpyWolf hereby excludes all liability and responsibility, and neither you nor any other person shall have any claim against SpyWolf, for any amount or kind of loss or damage that may result to you or any other person (including without limitation, any direct, indirect, special, punitive, consequential or pure economic loss or damages, or any loss of income, profits, goodwill, data, contracts, use of money, or business interruption, and whether in delict, tort (including without limitation negligence), contract, breach of statutory duty, misrepresentation (whether innocent or negligent) or otherwise under any claim of any nature whatsoever in any jurisdiction) in any way arising from or connected with this report and the use, inability to use or the results of use of this report, and any reliance on this report. The analysis of the security is purely based on the smart contracts, website, social media and team.
No applications were reviewed for security. No product code has been reviewed.